Codeus Practice

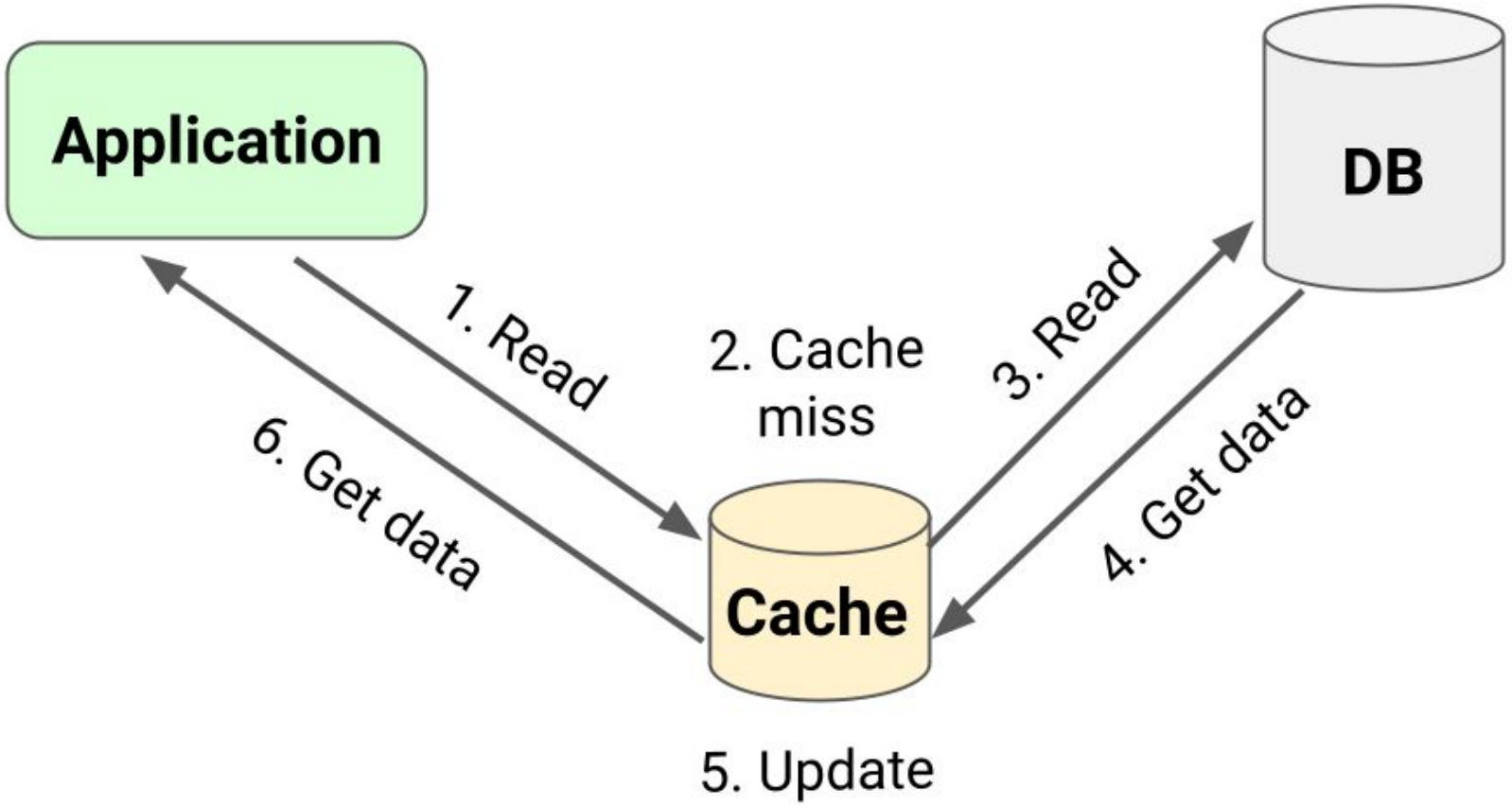
Write-through, write-back, read-through

About me

Name: Yelyzaveta Lubenets
Software Engineer @PrivatBank

Read-through

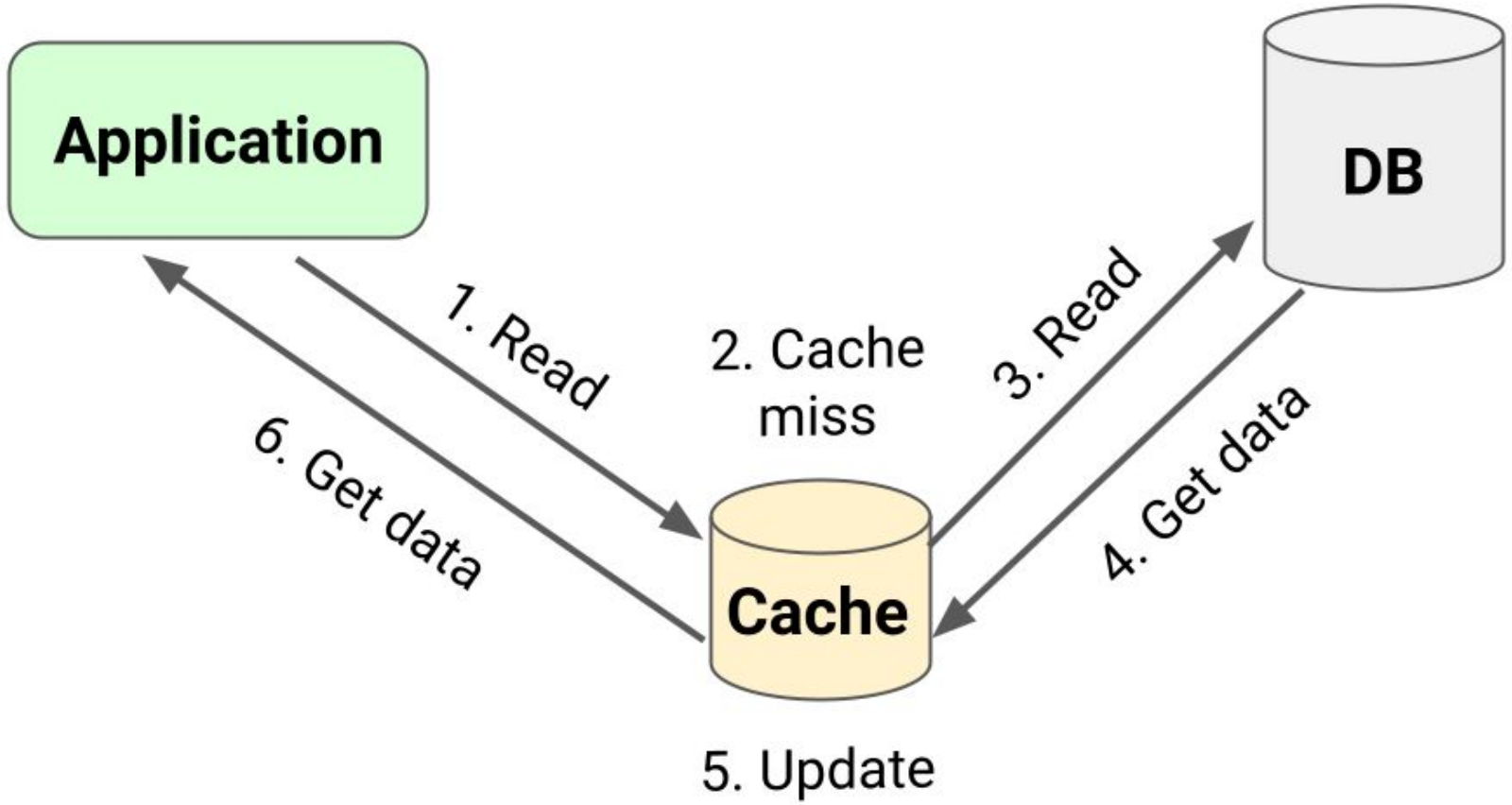
Application requests data from cache



Read-through

Application requests data from cache

Cache will first check if data is present

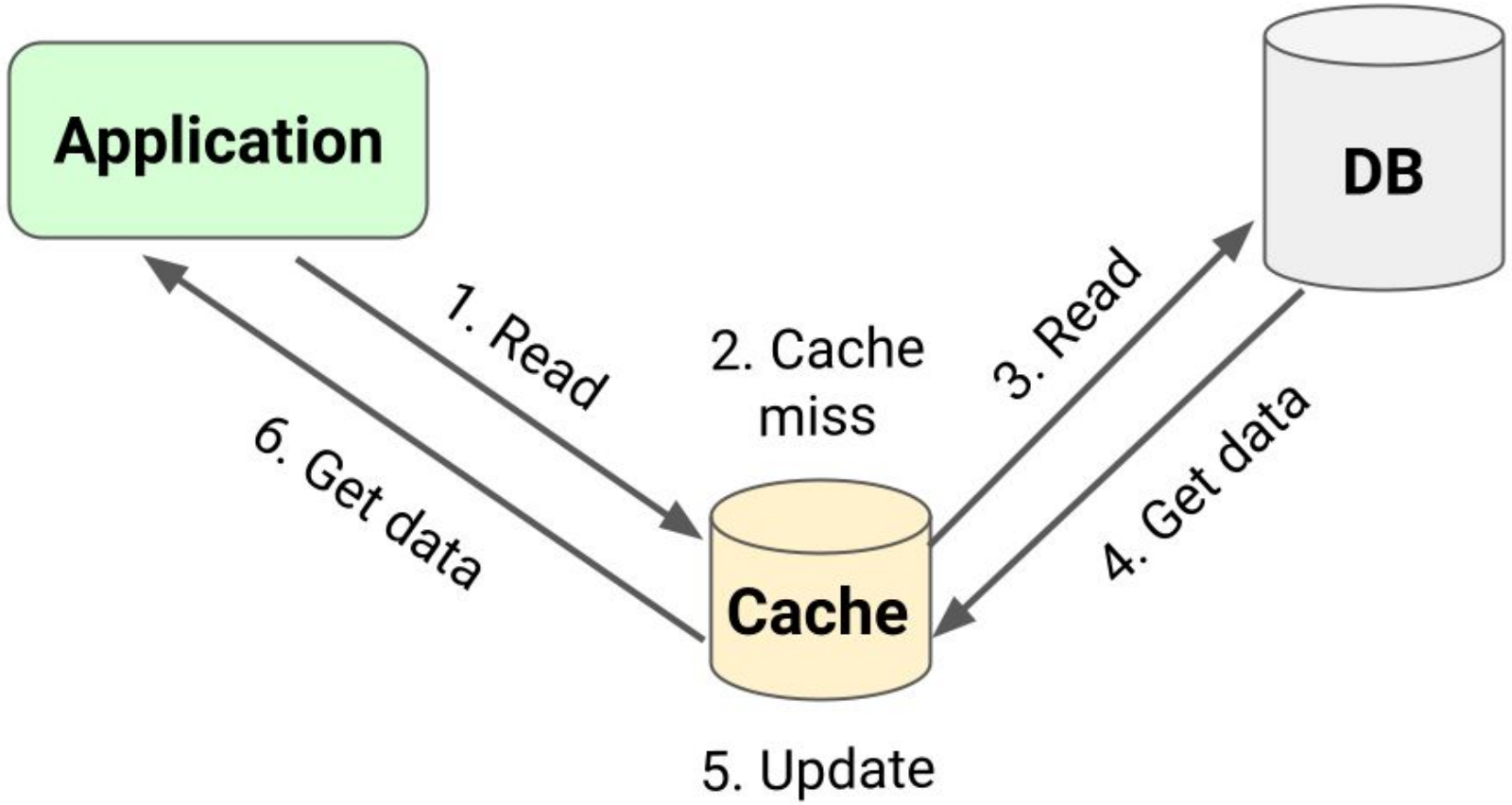


Read-through

Application requests data from cache

Cache will first check if data is present (Cache hit)

If yes, the data will be returned directly to the application

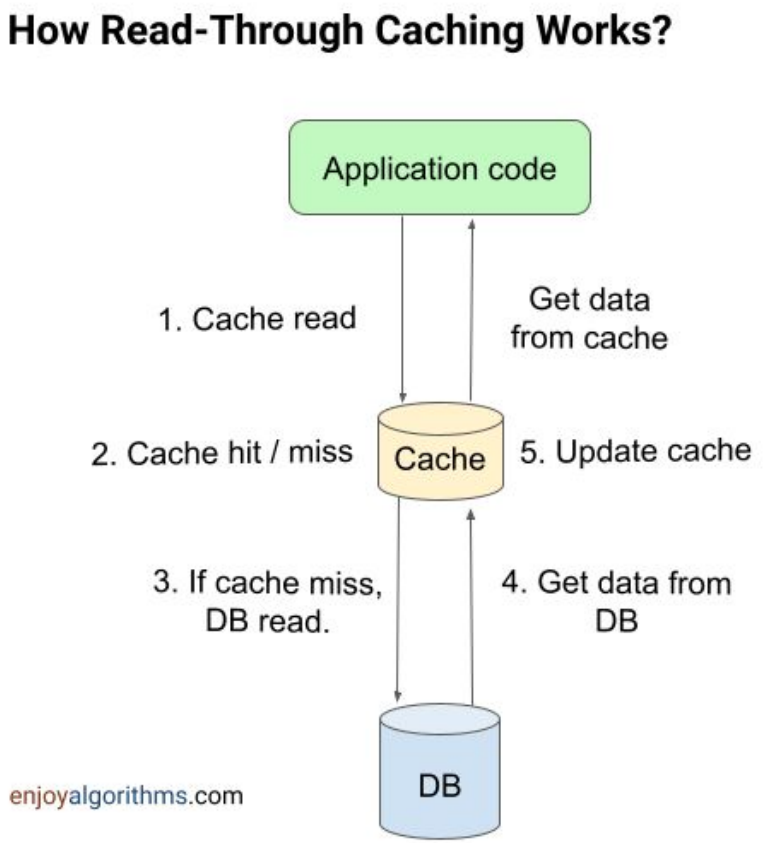


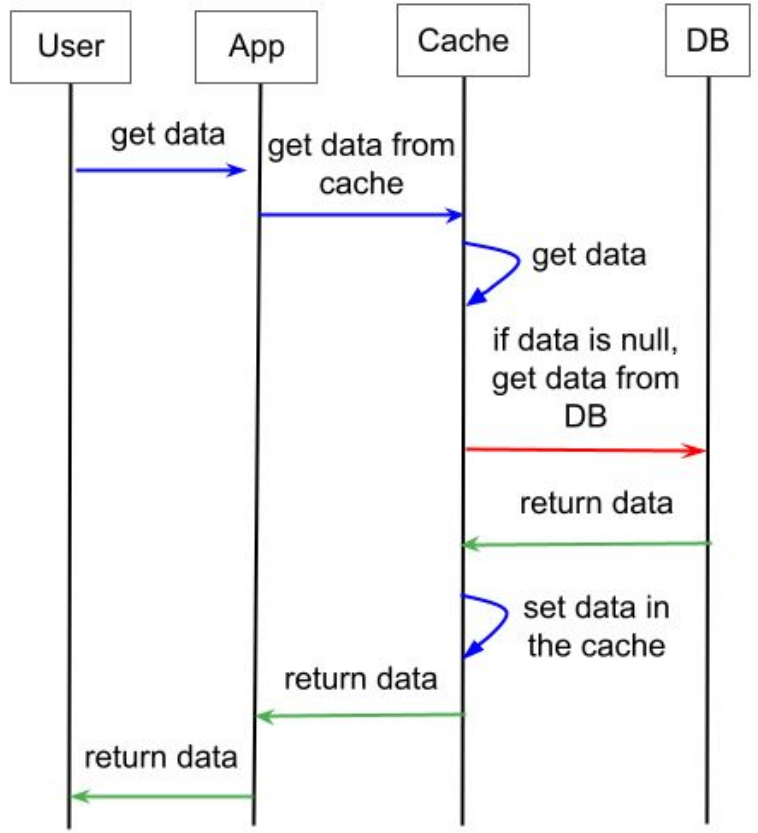
Read-through

Application requests data from cache

Cache will first check if data is present

If data is not present (Cache miss)





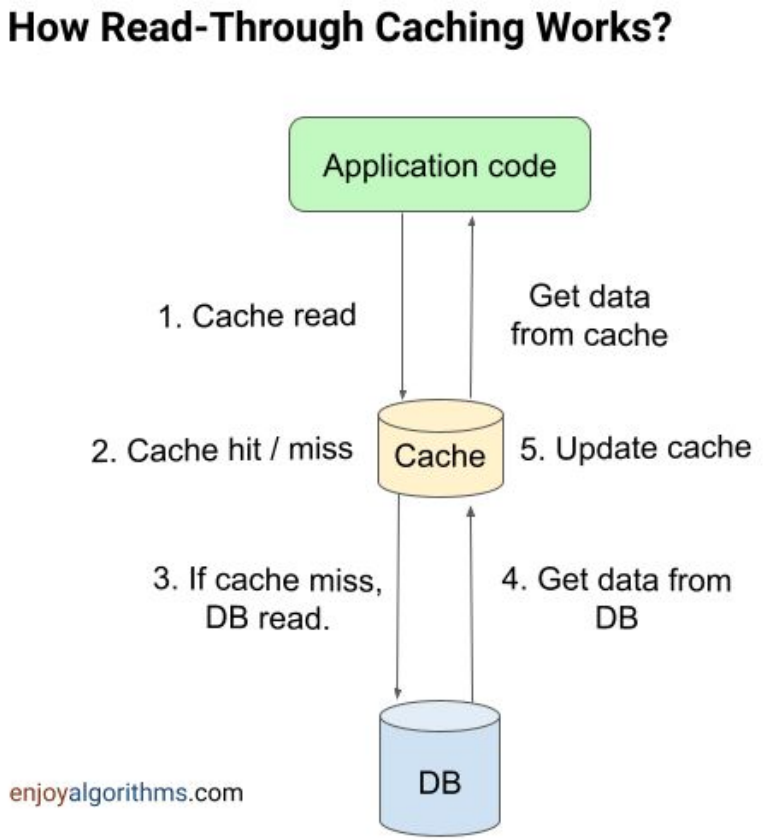
Read-through

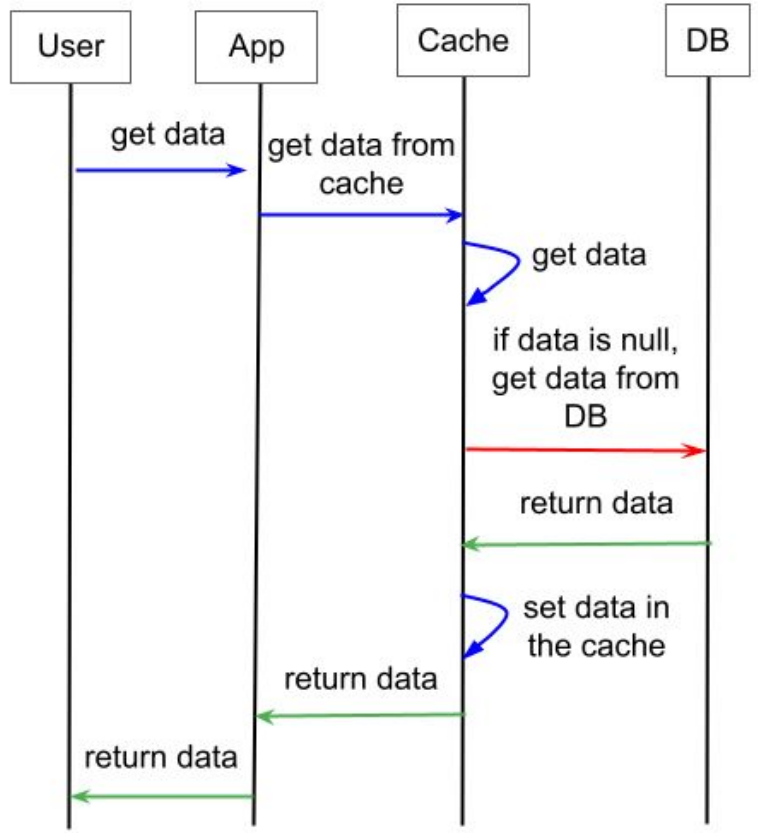
Application requests data from cache

Cache will first check if data is present

If data is not present (Cache miss)

Cache will act as a mediator and retrieve data from db





Read-through

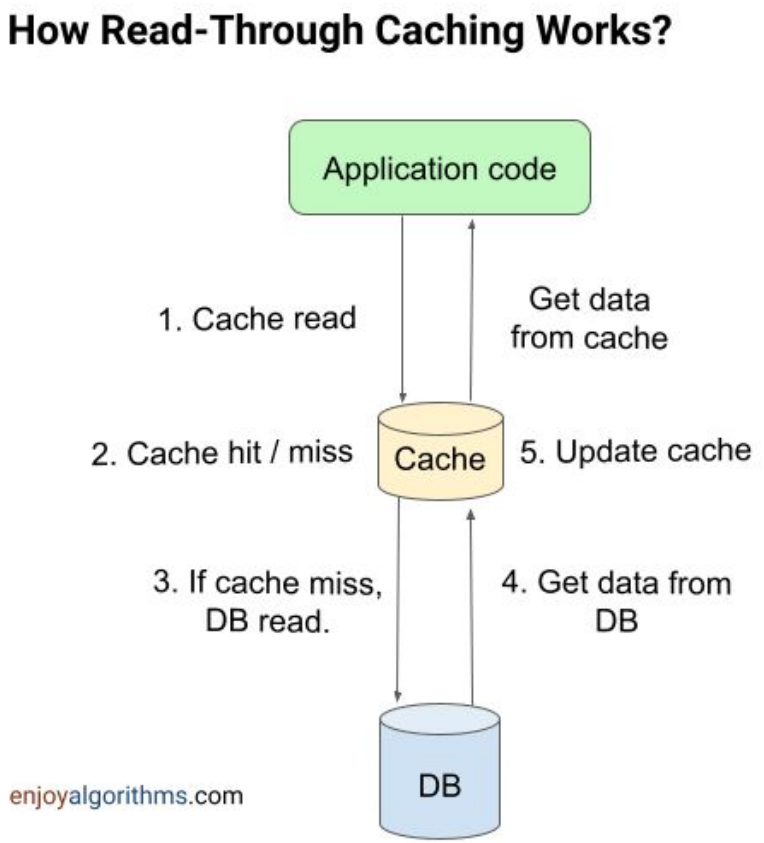
Application requests data from cache

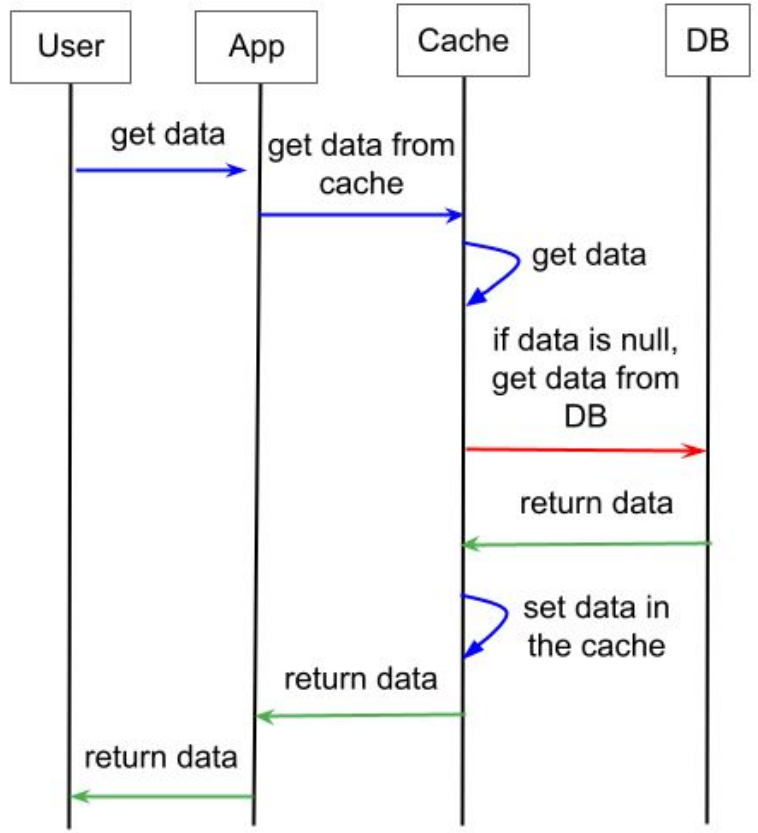
Cache will first check if data is present

If data is not present (Cache miss)

Cache will act as a mediator and retrieve data from db

ache will sto retrieved data and return it as a result of the original read request





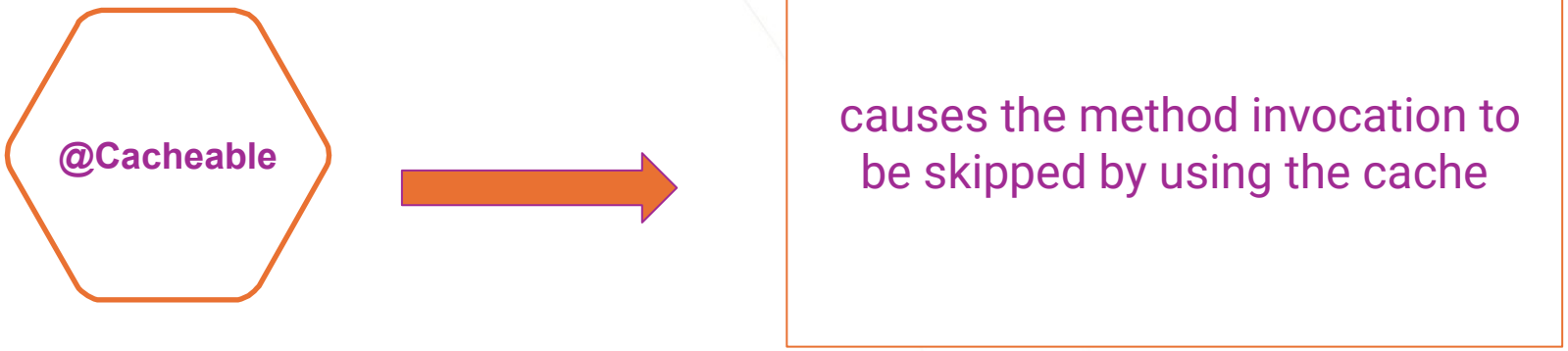
Read -through benefits

- Reduces read latency for frequently accessed data
- Reduces the number of requests to the original data source
- Could be configured to automatically reload data from database when it expires or when corresponding data changes in database

Read-through drawbacks

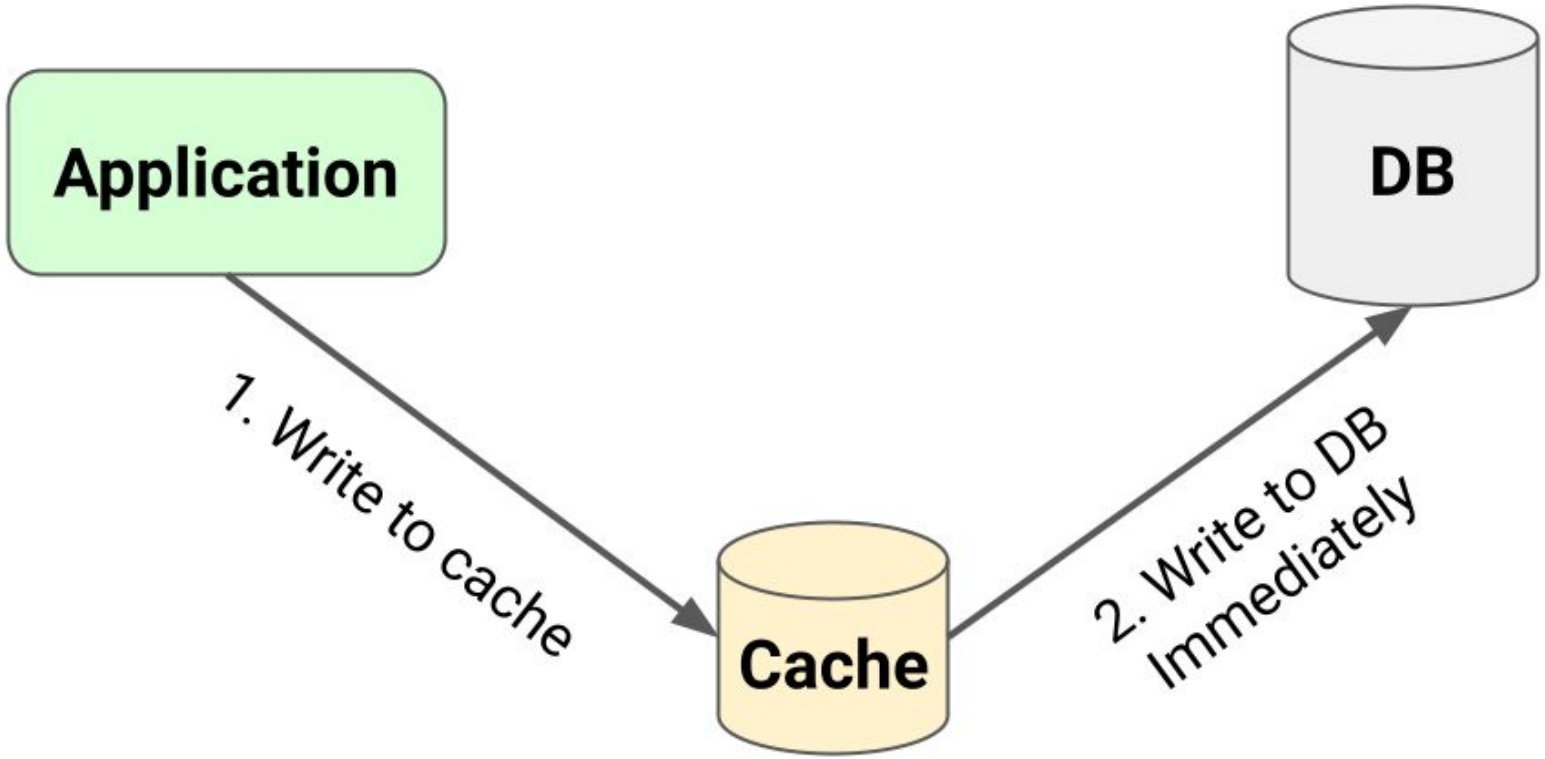
- New read request/ data will add extra latency
- Can lead to caching the data which will never access again
- Data can become inconsistent

Implementation



Write-through

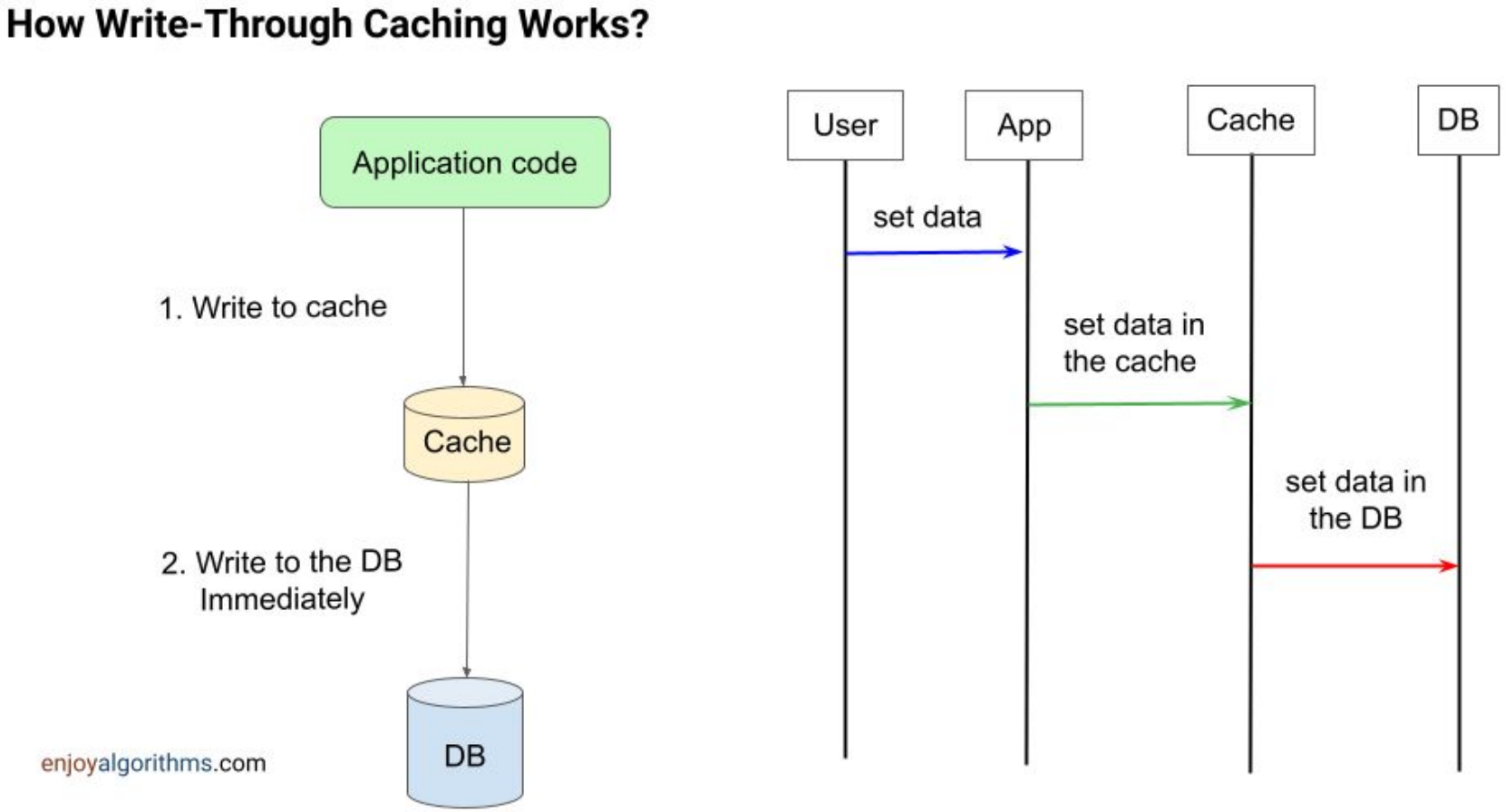
Application send the data needed to be add to the db



Write-through

Application send the data to be add to the db

Data will be first written to the cache

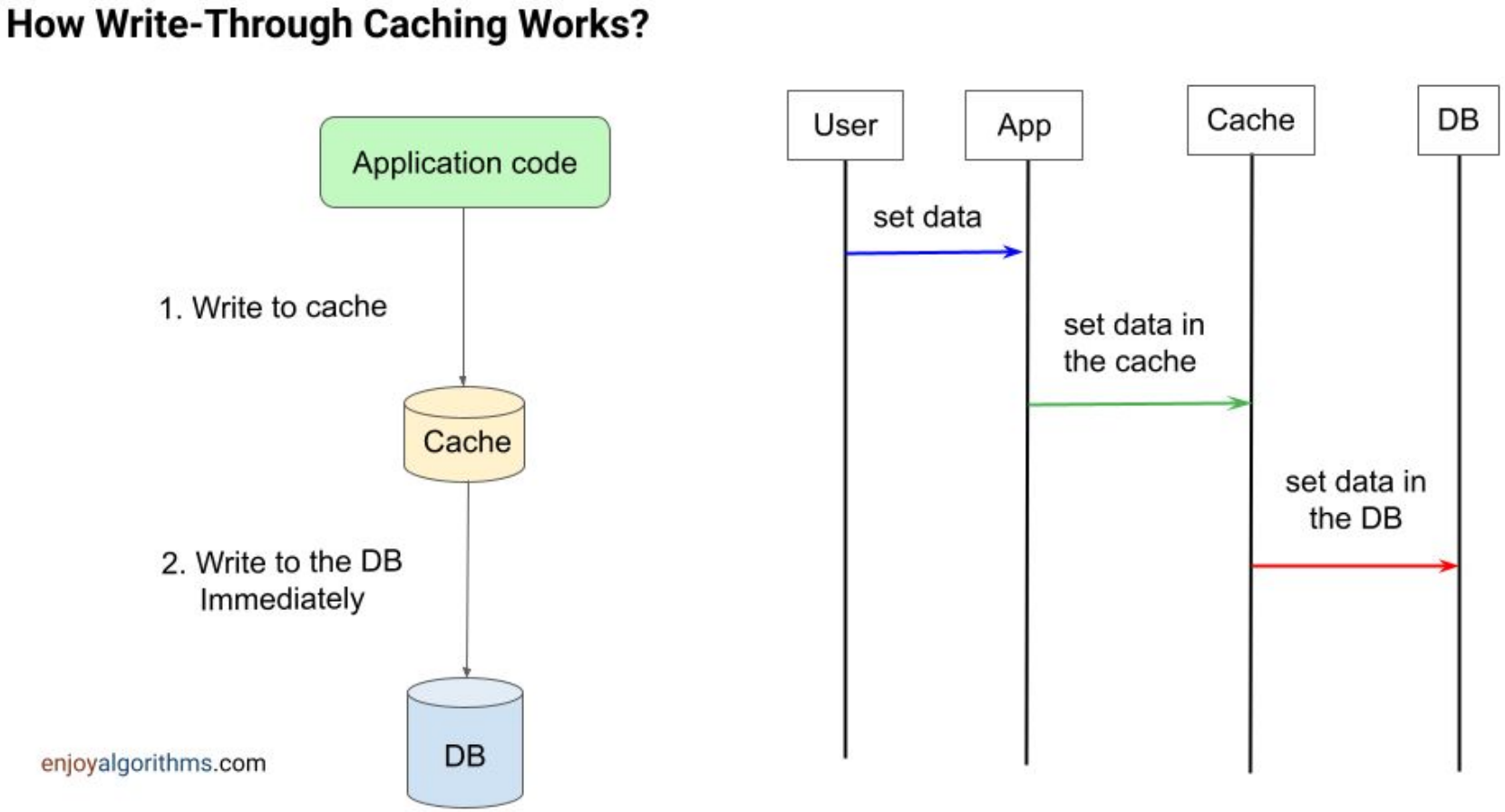


Write-through

Application send the data to be add to the db

Data will be first written to the cache

Data will be then written to the db immediately



Write -through benefits

Since all writes are immediately reflects in the database, all reads of the same data will get an up-to-date version

Reduces risk of data loss

Write-through drawbacks

Latency of write operation is very high

If system uses write-through cache for both read and write operation, a cache failure can cause severe issue

Why there is a need for cache eviction strategy?

In one way, it looks like there is no need for a cache eviction strategy in a write-through cache because cache is always consistent with the database. But practically, this is not the scenario! We still need to define TTL or other eviction strategies (LRU or LFU). Why? There are several reasons:

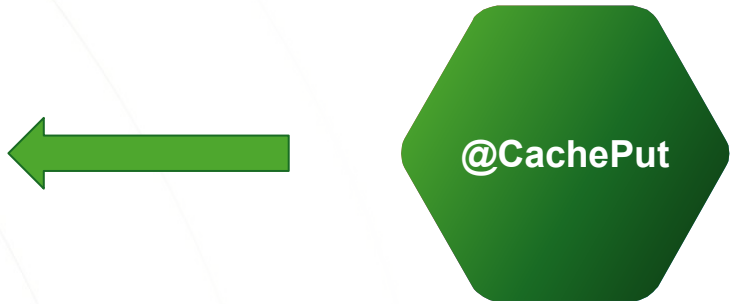
To avoid cluttering up cache with extra data and ensure that cache contains the most frequently accessed data

Storing all data indefinitely could lead to inefficient cache usage.

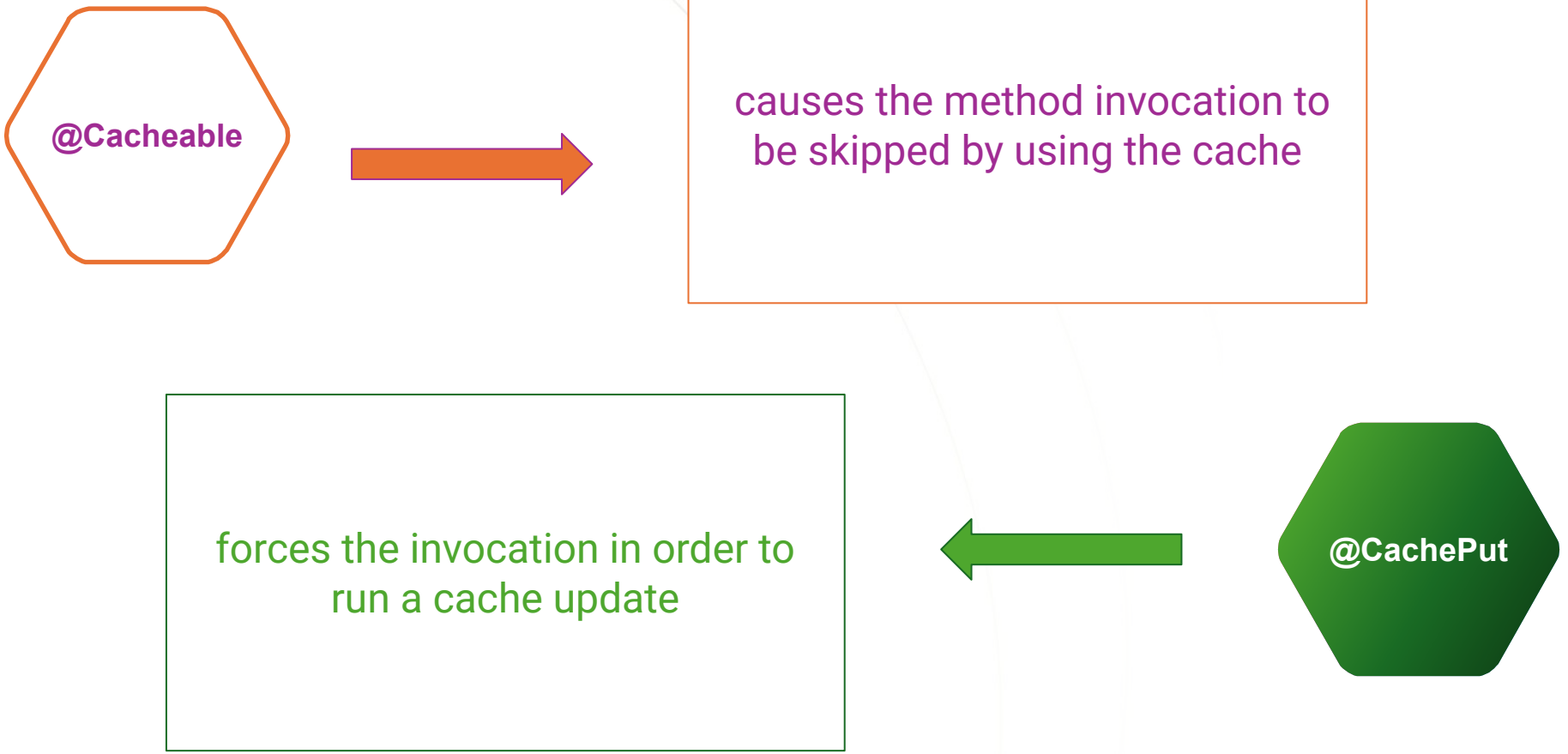
The need to define a proper eviction strategy to ensure updated data in the cache.

Implemenation

forces the invocation in order to run a cache update

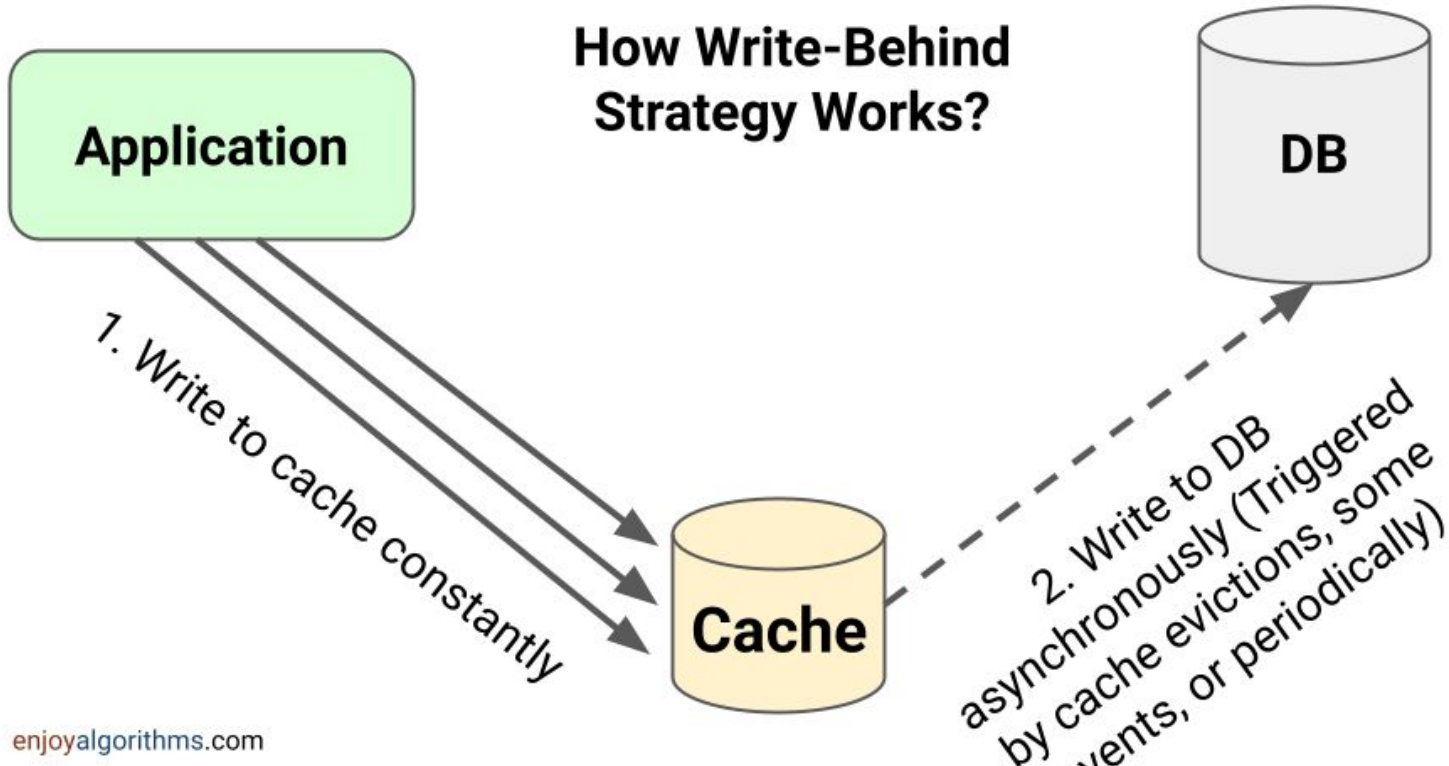


@Cacheable vs @CachePut



Write-behind (write-back)

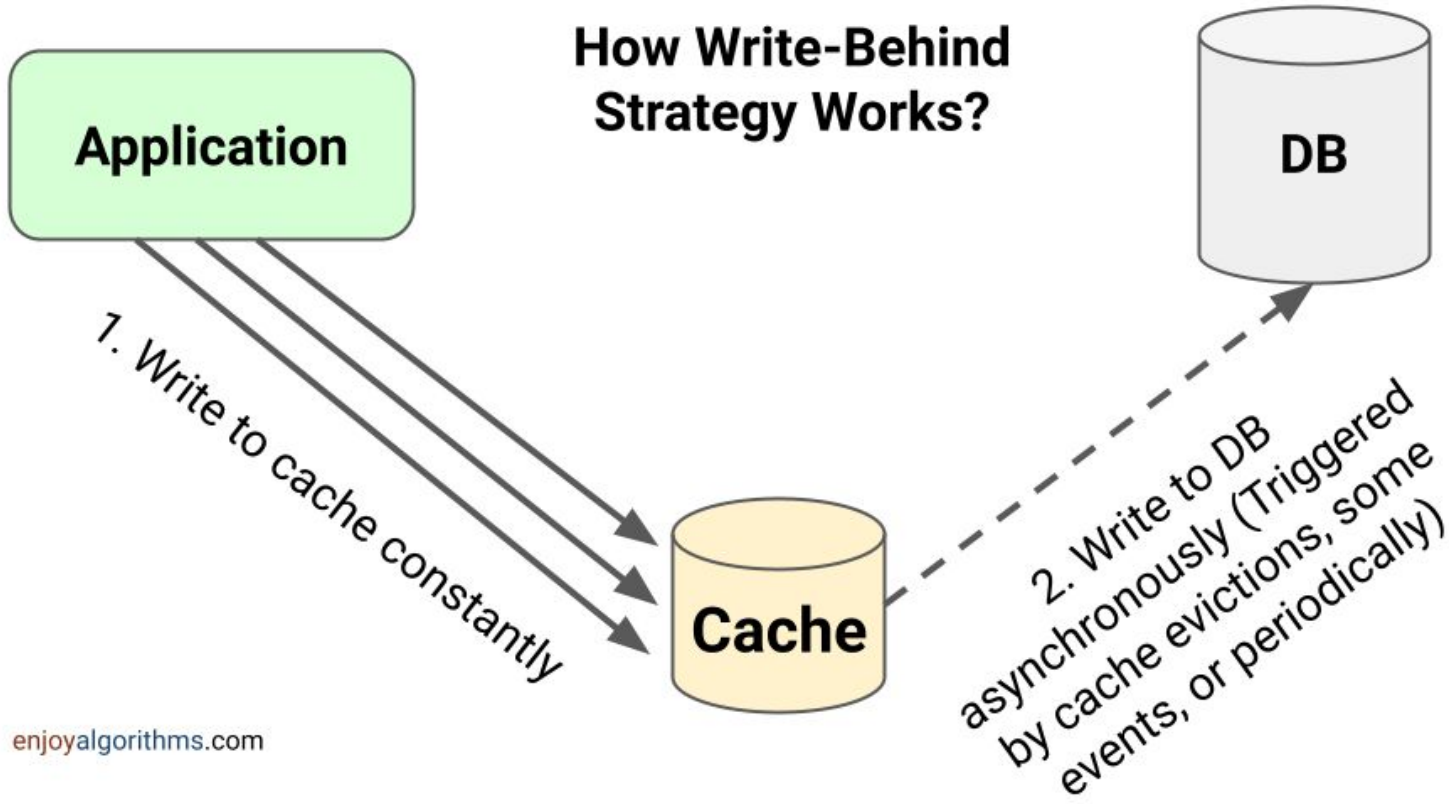
Application send the data needed to be updated in the db



Write-behind (write-back)

Application send the data to be updated in the db

Data will be first written to the cache

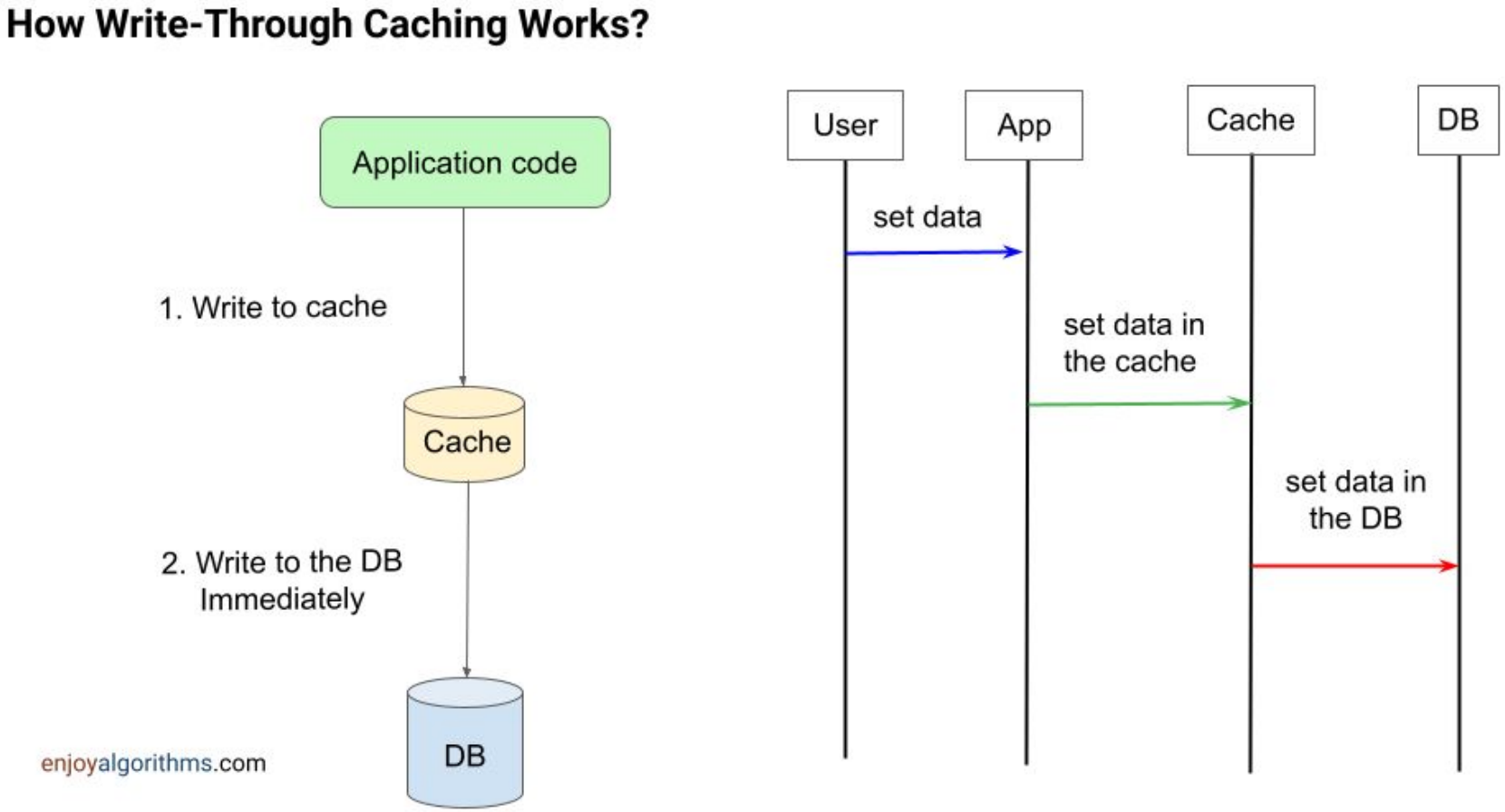


Write-behind (write-back)

Application send the data to be updated in the db

Data will be first written to the cache

Data will be then written to the db asynchronously



Write -behind (write-back) benefits

- Improve performance of write operation
- Reduce strain on the cache
- Reduce workload of the database

Write-behind (write back) drawbacks

- Delay between write to cache and to the database create time where data in the database is inconsistent
- If there is a cache failure, recently written data can be permanently lost
- If defer write to the database will fail, data will be lost

Approaches to offload database

Using rate limiting

When a lot of write requests come in all at once, it can overwhelm the database. To avoid this, we can use rate limiting in a write-behind cache to put a cap on the number of write requests the database can handle per second or per minute.

Using batching and coalescing technique

We can use batching and coalescing techniques in the write-behind cache to reduce the number of write requests. Batching combines multiple write operations into a single write while coalescing consolidates multiple updates on the same data into a single update.

Using time shifting

Databases can experience "rush hours" when lots of data are being written or modified simultaneously. So time shifting is an idea to strategically move the process of writing data to less busy times or time intervals other than peak usage times. This will allow the system to avoid becoming overwhelmed during high contention periods.

Write-through VS Write-behind

Write-through

- Maintenance consistency between cache and db
- Strong data durability
- High Latency for write
- Well suited for highly-sensitive data / scenarios where data consistency and integrity are top priorities

Write-behind

- Can lead to temporary inconsistency between cache and db
- Potential risk of data loss
- Low Latency for write
- Good for non-critical data / scenarios where write-performance or write-heavy workloads are crucial

Thank you

- Author: Yelyzaveta Lubenets
- My LinkedIn: https://www.linkedin.com/in/yelyzaveta-lubenets-275312227/
- Date: October 2025
- Join Codeus community in Discord
- Join Codeus community in LinkedIn